

codehive

Python User Input

Interactive Programs and Data Collection

PYTHON INTERACTION MODULE

1. Introduction to User Input

In Python, we use the **input()** function to allow user interaction. This makes your programs dynamic rather than static.

When the `input()` function is called, the program execution **pauses**. It waits for the user to type something and press the Enter key.

Basic Structure:

```
print("Enter your name:")  
name = input()  
print(f"Hello {name}")
```

2. The Prompt Parameter

Instead of using a separate `print()` statement, you can pass a string to the `input()` function. This string acts as a **prompt**.

```
# The prompt appears on the same line as the cursor  
name = input("Enter your name: ")
```

Visual Advantage

Using the prompt parameter keeps the terminal clean and ensures the user knows exactly what data is expected without extra line breaks.

3. Sequence of Inputs

Python executes code line by line. If you have multiple input calls, the program will stop and wait at each one sequentially.

Program Execution

1. Wait for 'Name' → Enter
2. Wait for 'Age' → Enter
3. Continue to Logic...

This allow you to build complex questionnaires or setup wizards effortlessly.

4. Handling Data Types

CRITICAL NOTE: The `input()` function always returns data as a **String** (str). Even if the user types a number like 45, Python sees it as "45".

Attempting to do math with raw input will cause a `TypeError`.

```
age = input("Enter age: ")  
# This will fail:  
# print(age + 10)
```

5. Input Casting

To use numerical input, you must "cast" or convert the string into an `int` or `float`.

Example: Square Root Calculator

```
import math
x = input("Enter a number: ")

# Convert string to float for math operations
y = math.sqrt(float(x))

print(f"The square root is {y}")
```

6. Multiple Inputs Example

You can create interactive "Mad Libs" or stories by gathering various strings from the user.

```
animal = input("Favorite animal: ")
color = input("Favorite color: ")
num = input("Favorite number: ")

print(f"Do you want a {color} {animal} with {num} legs?")
```

By using f-strings, you can easily inject these user variables into your output messages.

7. Input Validation Flow

What happens if a user enters "ABC" when you ask for a number? The program crashes. To prevent this, we use **Validation**.

The Validation Loop Strategy:

- Start an infinite `while` loop.
- Ask for input.
- Try to convert the input.
- If successful, `break` the loop.
- If failed, catch the error and ask again.

8. Building a Robust Input

Here is the professional way to handle user input to ensure your program never crashes due to bad data.

```
active = True
while active:
    val = input("Enter a number: ")
    try:
        val = float(val)
        active = False # Success! Exit loop
    except ValueError:
        print("Invalid input! Use digits only.")

print(f"Validated number: {val}")
```

9. Practical Exercises

Knowledge Check

Question: What is the name of the method used to ask for user input?

☐ `prompt()`

☒ `input()`

☐ `user()`

Task: Write a script that asks for two numbers and prints their sum. (Remember to cast!)

10. Summary Checklist

User Input Mastery Checklist

- ✓ Always use `input()` for interactive scripts.
- ✓ Use a prompt string inside the function for better UX.
- ✓ Cast to `int()` or `float()` for math.
- ✓ Implement `try...except` to handle non-numeric input.
- ✓ Use f-strings to display the results back to the user.

codehive

Authorized Publication 2024